MILESTONE 1

Kelompok: Unisoft

Nomor Kelompok: G03

List Anggota Kelompok:

1. **Bayu Palamarta Wirawan - 13525085**
2. **Muhammad Fauzi Muharam - 13525091**
3. **Muhammad Pandu Pulunggana - 13525059**
4. **Kevin Lincoln Hutabarat - 13525101**

PROGRESS

1. COMPLETED TASKS

No	Task	Penanggung Jawab (NIM)
1	startGame	13525085
2	cekInfo	13525085
3	ambilKartu	13525091
4	lihatKartu	13525059
5	mainKartu	13525101
6	lihatCommand	13525101

2. ONGOING TASKS

No	Task	Penanggung Jawab (NIM)
1	draw_two	
2	wild_draw_four	
3	skip	

4	reverse	

3. UNSTARTED TASKS

No	Task	Penanggung Jawab (NIM)
1	tantang	
2	tangkap	
3	uni	
4	endGame	
5	saveGame	
6	loadGame	

RANCANGAN FAKTA

No	Fakta	Deskripsi
1.	`statusEfek(off).`	statusEfek digunakan untuk situasi ketika pemain terkena drawtwo atau wilddrawfour
2.	`get_element([Element|_], 0, Element).`	Permulaan fungsi get_element
3.	`input_pemain(-1, _).`	Basis input nama pemain
4.	`buat_list(_, 0, []).`	Basis buat_list
5.	`start_kartu(0).`	Basis start_kartu
6.	`count([],0).`	Basis count
7.	`cetakpemain(0).`	Basis cetakpemain
8.	`printKartu([], []).`	Basis printKartu

9.	`printKoma([]).`	Basis printKoma
10.	`tampilkanKartu([], [], _).`	Basis tampilkanKartu
11.	`kartu([0,1,2,3,4,5,6,7,8,9,'skip','reverse','drawtwo','wild','wilddrawfour'], ['merah','kuning','hijau','biru']).`	Menyatakan semua kemungkinan nilai kartu
12.	`get([Element|_], 0, Element).`	Basis get untuk file1
13	`ambil_7_kali([], [], 0).`	Basis ambil_7_kali
14	`top_card_sebelumnya(a,b).`	Basis top_card_sebelumnya untuk tantang
15	`cekList(X1, Y1, [X1|_], [Y1|_], 0):- !.` `cekList(_,_, [], [], -1):- !.`	Basis cekList
16	`insert_head(H, [], [H]).` `insert_head(H, T, [H|T]).`	Memasukan head dalam array
17	`removeListIdx([_|T1], [_|T2], T1, T2, 0).`	Basis removeListIdx

18	`validasiwarna('merah').`
	`validasiwarna('biru').`
	`validasiwarna('kuning').`
	`validasiwarna('hijau').`
		Memeriksa 4 kemungkinan warna kartu

19	`cekAngka(_,_).`
	`cekSkip(_,_).`
	`cekReverse(_,_).`
	`cekWild(_).`
	`cekDrawTwo(_,_).`
	`cekDrawFour(_).`
		Basis pengecekan kartu

20	`listkosong([],[],1).`
	`listkosong([_|_],[],0).`
	`listkosong([],[_|_],0).`
		Menentukan apakah 2 list itu kosong

	`listkosong([_|_],[_|_],0).`	
21	`jumlahElemenList([],0).`	Basis jumlahElementList
22	`ceksalah(1).`	Basis ceksalah
23	`cekTantang(_,Count,Count).`	Basis cekTantang
24	`cekUni(_,Count,Count).`	Basis cekUni
25	`start:-` `  cekTangkap(0,0,5).`	Basis start

RANCANGAN RULE

No	Rule	Deskripsi
1.	`get_element([_|Tail], Index, Element) :- Index > 0, NI is Index - 1, get_element(Tail, NI, Element).`	Mencari elemen di list dengan index tertentu

2.	`cek_pemain(X, X) :-`
	`  X >= 2,`
	`  X =< 4, !.`
		Kondisi jumlah pemain sesuai

3.	`cek_pemain(_, Y) :-`
	`  write('Mohon masukan angka antara 2-4: '),`
	`  read(Z), nl,`
	`  cek_pemain(Z, Y).`
		Kondisi jumlah pemain tidak sesuai

4.	`cek_nama(X, X) :-`
	`  \+ listpemain(_, X, _, _), !.`
	`cek_nama(_, Y) :-`
	`  write('Nama sudah digunakan. Masukan nama lain: '),`
		Mengecek apakah ada nama duplikat

	`read(Z), nl,` `cek_nama(Z, Y).`	

5.	`input_pemain(X, Y) :-`
	`X >= 0,`
	`write('Masukan nama pemain '),`
	`Y1 is Y - X,`
	`write(Y1),`
	`write(': '),`
	`read(Z),`
	`cek_nama(Z, Z1),`
	`assertz(listpemain(Y1, Z1, [], [])),`
	`X1 is X - 1,`
	`input_pemain(X1, Y).`
		Memasukan nama pemain dalam dynamic

6.	`buat_list(X, X, [X|T]) :-`
	`X > 0,`
	`X1 is X - 1,`
	`buat_list(X1, X1, T).`
		Membuat list baru untuk urutan pemain

7.	`start_kartu(XValid):-`
	`XValid > 0,`
	`listpemain(XValid, Nama, _, _),`
	`ambil_7_kali(KartuBaru, WarnaBaru, 7),`
	
	`retract(listpemain(XValid, Nama, _, _)),`
	
	`assertz(listpemain(XValid, Nama, KartuBaru, WarnaBaru)),`
	`X1 is XValid - 1,`
	`start_kartu(X1).`
		Memilih 7 kartu bebas untuk setiap pemain

8.	printurutan([T]) :- listpemain(T, Nama, _, _), write(Nama), write('.'), nl.	Print nama pemain giliran terakhir
9.	printurutan([H|T]):- listpemain(H, Nama, _, _), write(Nama), write(' - '), printurutan(T).	Print nama pemain giliran pertama hingga kedua terakhir
10.	startGame :- write('Masukan jumlah pemain: '), read(X), cek_pemain(X, XValid), nl, X1 is XValid - 1, input_pemain(X1, XValid), assertz(jumlahPemain(XValid)), buat_list(XValid, XValid, R), permutation(R, R1), !, assertz(urutanGiliran(R1)), write('Urutan pemain: '), printurutan(R1), start_kartu(XValid), write('Setiap pemain mendapatkan 7 kartu acak'), nl, ambil_kartu_top(A, B), assertz(top_card(A, B)), /*print */	Memulai permainan

	write('Kartu discard top: '), write(B), write('-'), write(A), nl, write('Giliran '), get_element(R1, 0, C), listpemain(C, N, _, _), write(N), write('.'),nl.	
11.	count([_|T], X) :- count(T, X1), X is X1 + 1.	Banyak anggota dalam array
12.	cetakpemain(X):- X > 0, X1 is X - 1, cetakpemain(X1), listpemain(X, Y, Z, _), write('Nama pemain '), write(X), write(': '), write(Y), nl, count(Z, N), write('Jumlah kartu : '), write(N), nl, nl.	Print semua pemain untuk cekInfo
13.	cekInfo :- top_card(X, Y), !, write('Kartu discard top: '), write(X), write('-'), write(Y), write(.),nl,nl, write('Urutan pemain: '), urutanGiliran(Z), printurutan(Z), nl, nl, jumlahPemain(P), cetakpemain(P).	Mengecek jumlah kartu tiap orang
14.	ambilKartu:-	Ambil kartu apabila terkena drawtwo

	/*liat kartu paling atas, kalo +2, ambil 2 kartu. perlu akses urutan turn dan tumpukan paling atas*/ statusEfek(on), urutanGiliran([X|_]), listpemain(X, _, _, _), top_card(Kartu, _), Kartu = drawtwo, !, ambilKartu(X, 2).	
15.	ambilKartu:- /*jika +4 maka ambil 4 kartu*/ statusEfek(on), urutanGiliran([X|_]), listpemain(X, _, _, _), top_card(Kartu, _), Kartu = wilddrawfour, !, ambilKartu(X, 4).	Ambil kartu apabila terkena wilddrawfour
16.	ambilKartu:- /*jika bukan +2/+4 maka ambil 1 kartu*/ urutanGiliran([X|_]), listpemain(X, _, _, _), ambilKartu(X, 1).	Ambil kartu biasa
17.	ambilKartu(Id, JumlahKartu) :- listpemain(Id, Nama, K_Lama, W_Lama), ambilKartu(JumlahKartu, K_Baru, W_Baru), write(Nama), write(' mendapatkan '), write(JumlahKartu), write('	Ambil kartu sesuai id pemain dan jumlah kartu yang diambil

	dengan rincian:'),
	    printKartu(K_Baru, W_Baru), nl,
	    append(K_Lama, K_Baru, K_Total),
	    append(W_Lama, W_Baru, W_Total),
	    retract(listpemain(Id, Nama, K_Lama, W_Lama)),
	    assertz(listpemain(Id, Nama, K_Total, W_Total)),
	    matikan_status_efek,
	    putarGiliran,
	
	urutanGiliran(UrutanBaru),
	    write('Urutan pemain: '),
	    printurutan(UrutanBaru), nl,
	    UrutanBaru = [NextId|_],
	    listpemain(NextId, NamaNext, _, _),
	    write('Giliran '), write(NamaNext), write('.'), nl.	

18.	matikan_status_efek :-
	    statusEfek(on),
	    retract(statusEfek(on)),
	    assertz(statusEfek(off)), !.
		Menonaktifkan statusEfek supaya pemain setelah pemain yang terkena drawtwo/wilddrawfour tidak harus mengambil kartu

19.	ambilKartu(0, [], []) :- !.
	ambilKartu(N, [K|SisaK], [W|SisaW]) :-
	    N > 0,
	    ambil_kartu_acak(K, W),
	    N1 is N - 1,
	    ambilKartu(N1, SisaK,
		Ambil kartu acak untuk warna dan jenis

	SisaW).	
20.	printKartu([K|SisaK], [W|SisaW]) :- write(W), write('-'), write(K), printKoma(SisaK), printKartu(SisaK, SisaW).	Print kartu untuk ambilKartu
21.	printKoma([_|_]) :- write(', ').	Mencetak koma untuk printKartu
22.	putarGiliran :- retract(urutanGiliran([H|T])), append(T, [H], UrutanBaru), assertz(urutanGiliran(UrutanBaru)).	Menukar giliran setelah giliran orang pertama selesai
23.	lihatKartu :- urutanGiliran([IdPemain|_]), listpemain(IdPemain, _, ListKartu, ListWarna), write('Berikut kartu yang anda miliki.'), nl, tampilkanKartu(ListKartu, ListWarna, 1).	Memperlihatkan kartu milik pemain giliran saat ini
24.	tampilkanKartu([Kartu|TK], [Warna|TW], N) :- write(N), write('. '), write(Warna), write('-'),	Menampilkan kartu untuk lihatKartu

	write(Kartu), nl, N1 is N + 1, tampilkanKartu(TK, TW, N1).	
25	get([_|Tail], Index, Element) :- Index > 0, NI is Index - 1, get(Tail, NI, Element).	Mendapatkan elemen tertentu di list dalam file1
26	ambil_kartu_acak(X, Y):- random(0, 15, P), kartu(A, _), get(A, P, X), X = 'wild', !, Y = 'hitam'.	Ambil kartu acak jika mendapatkan wild
27	ambil_kartu_acak(X, Y):- random(0, 15, P), kartu(A, _), get(A, P, X), X = 'wilddrawfour', !, Y = 'hitam'.	Ambil kartu acak jika mendapatkan wilddrawfour
28	ambil_kartu_acak(X, Y) :- random(0, 15, P), random(0, 4, Q), kartu(A, B), get(A, P, X), get(B, Q, Y).	Ambil kartu acak untuk kasus lainnya
29	ambil_kartu_top(X,Y) :- random(0, 10, P), random(0, 4, Q), kartu(A, B), get(A, P, X), get(B, Q, Y).	Mengambil kartu paling atas untuk memulai permainan
30.	ambil_7_kali([H|T], [A|B], N) :- N > 0, ambil_kartu_acak(H, A), N1 is N - 1, ambil_7_kali(T, B, N1).	Mengambil kartu 7 kali untuk memulai pertandingan.
31	cekList(X1, Y1, [_|T1], [_|T2], Idx):- I is Idx+1,	Memeriksa index dari elemen dalam suatu array

	cekList(X1, Y1, T1, T2, I).	
32	removeListIdx([H1|T1], [H2|T2], X, Y, Idx):- Idx>0, I is Idx-1, removeListIdx(T1, T2, X1, Y1, I), insert_head(H1,X1,X), insert_head(H2,Y1,Y).	Menghapus elemen dari array
33	cekKartuValid(K):- urutanGiliran(R1), get_element(R1, 0, C), listpemain(C, _, X, _), jumlahElemenList(X,Sum), S is Sum+1, K<S, K>=1.	Memeriksa apakah sebuah kartu valid secara angka pilihan
34	pilihWarna(Y, _):- \+validasiwarna(Y), nl, write('Warna tidak valid, input lagi:'), read(Y1), pilihWarna(Y1, _). pilihWarna(Y, Y):- validasiwarna(Y).	Memilih warna apabila memainkan kartu wild
35	cekAngka(X1,Y1):- number(X1), X1>=0, X1<10, putarGiliran, retract(top_card(_,_)), assertz(top_card(X1,Y1)),	Permainan kartu angka biasa

	urutanGiliran(R2), get_element(R2, 0, C2), listpemain(C2, N2, _, _), write('Giliran '), write(N2), nl.	
36	cekSkip(X1,Y1):- X1 == 'skip', write('Pemain berikutnya kehilangan giliran.'), nl, nl, putarGiliran, putarGiliran, retract(top_card(_,_)), assertz(top_card(X1,Y1)), urutanGiliran(R2), get_element(R2, 0, C2), listpemain(C2, N2, _, _), write('Giliran '), write(N2), nl.	Permainan kartu skip
37	cekReverse(X1,Y1):- X1 == 'reverse', urutanGiliran([H|T]), reverse_list([H|T],R), retract(urutanGiliran(_)), assertz(urutanGiliran(R)), retract(top_card(_,_)), assertz(top_card(X1,Y1)), urutanGiliran(R2), get_element(R2, 0, C2), listpemain(C2, N2, _, _), write('Giliran '), write(N2), nl.	Permainan kartu reverse
38	cekWild(X1):- X1 == 'wild',	Pengecekan kartu wild

	write('Pilih warna: '), read(Y2), nl, pilihWarna(Y2,Y3), putarGiliran, retract(top_card(_,_)), assertz(top_card(X1,Y3)), urutanGiliran(R2), get_element(R2, 0, C2), listpemain(C2, N2, _, _), write('Giliran '), write(N2), nl.	
39	cekDrawTwo(X1,Y1):- X1 == 'drawtwo', retract(statusEfek(_)), assertz(statusEfek(on)), putarGiliran, retract(top_card(_,_)), assertz(top_card(X1,Y1)), ambilKartu.	Permainan kartu drawtwo
40	cekDrawFour(X1):- X1 == 'wilddrawfour', write('Pilih warna: '), read(Y2), nl, pilihWarna(Y2,Y3), retract(statusEfek(_)), assertz(statusEfek(on)), putarGiliran, retract(top_card(_,_)), assertz(top_card(X1,Y3)), urutanGiliran(R2), get_element(R2, 0, C2), listpemain(C2, N2, _, _), write('Giliran '), write(N2), nl. cekDrawFour(_).	Permainan kartu wilddrawfour

41	```
	cekKartu(A, B, _, Y1):-
	    number(A),
	    A>=0,
	    A<10,
	    B == Y1,
	    !.
	cekKartu(A, _, X1, _):-
	    number(A),
	    number(X1),
	    A>=0,
	    A<10,
	    A =:= X1,
	    !.
	
	cekKartu(A, _, X1, _):-
	    A == 'skip',
	    A == X1,
	    !.
	cekKartu(A, B, _, Y1):-
	    A == 'skip',
	    B == Y1,
	    !.
	
	cekKartu(A, _, X1, _):-
	    A == 'reverse',
	    A == X1,
	    !.
	cekKartu(A, B, _, Y1):-
	    A == 'reverse',
	    B == Y1,
	    !.
	
	cekKartu(A, B, X1, Y1):-
	    A == 'drawtwo',
	    A \== X1,
	    B == Y1,
	    !.
	
	cekKartu(A, B, X1, Y1):-
	``
		Mengecek apakah kriteria kartu sesuai untuk dimainkan

	A == 'wild', A \== X1, B == Y1, !. cekKartu(A, B, X1, Y1):- A == 'wilddrawfour', A \== X1, B == Y1, !. cekKartu(A, _, X1, _):- X1 == 'wild', A \== 'wild', !. cekKartu(A, _, X1, _):- X1 == 'wilddrawfour', A \== 'wilddrawfour', !.	
42	mainkanKartu(_):- statusEfek(on), write('Perintah tidak valid.'), !, nl. mainkanKartu(K):- \+cekKartuValid(K), write('Kartu tidak valid!'), !, nl. mainkanKartu(K):- urutanGiliran(R1), top_card(A, B), get_element(R1, 0, C), listpemain(C, _, X, Y),	Memainkan 1 kartu berdasarkan pilihan pengguna

```
    cekKartuValid(K),
    Idx is K-1,
    get_element(X,Idx,X1),
    get_element(Y,Idx,Y1),
    \+cekKartu(A, B, X1, Y1),
    !,
    write('Kartu tidak
valid!'),
    nl.
mainkanKartu(K):-
    urutanGiliran(R1),
    top_card(A, B),
    get_element(R1, 0, C),
listpemain(C, N, X, Y),
    cekKartuValid(K),
    Idx is K-1,
    get_element(X,Idx,X1),
    get_element(Y,Idx,Y1),
    cekKartu(A, B, X1, Y1),
    nl,
    write(N), write('
memainkan kartu: '),
    write(Y1), write('-'),
write(X1),
    nl, nl,
    removeListIdx(X, Y, X2,
Y2, Idx),

retract(listpemain(C,N,_,_)),

assertz(listpemain(C,N,X2,Y2)
),
    cekAngka(X1,Y1),
    cekSkip(X1,Y1),
    cekReverse(X1,Y1),
    cekDrawTwo(X1,Y1),
    cekWild(X1),
    cekDrawFour(X1),
```

	retract(top_card_sebelumnya(_,_)), assertz(top_card_sebelumnya(A,B)).	

43	jumlahElemenList([_|T],Sum):-
	    jumlahElemenList(T,S1),
	    Sum is S1+1.
		Mencari jumlah element di sebuah list

44	cekSemuaKartu(_,_,_,_,N,N):-!, ceksalah(0).
	cekSemuaKartu(A,B,X,Y,Count,N):-
	    get_element(X,Count,X1),
	    get_element(Y,Count,Y1),
	    \+cekKartu(A,B,X1,Y1),
	    C1 is Count+1,
	
	cekSemuaKartu(A,B,X,Y,C1,N).
	cekSemuaKartu(A,B,X,Y,Count,_):-
	    get_element(X,Count,X1),
	    get_element(Y,Count,Y1),
	    cekKartu(A,B,X1,Y1),
	    !
		Memeriksa apakah ada setidaknya satu kartu yang bisa dimainkan

45	cekMain(_,_,_,_,Out):-
	    statusEfek(on),
	    !,
	    Out is 1.
	cekMain(_,_,X,Y,Out):-
	    listkosong(X,Y,1),
	    !,
	    Out is 1.
	cekMain(A,B,X,Y,Out):-
	    listkosong(X,Y,0),
	    jumlahElemenList(X,N),
		Menentukan apakah aksi utama mainkanKartu bisa dijalankan untuk lihatCommand

	cekSemuaKartu(A,B,X,Y,0,N), !, write('1. mainkanKartu'), nl, Out is 2. cekMain(_,_,_,_,Out):- Out is 1.	
46	cekTantang(A, Count, Out):- A == 'wilddrawfour', statusEfek(on), !, write(Count), write('. tantang'), nl, Out is Count+1.	Memeriksa apakah aksi tantang bisa dilakukan
47	cekUni(X,Count, Out):- jumlahElemenList(X,Sum), Sum =:= 2, !, write(Count), write('. uni'), nl, Out is Count+1.	Memeriksa apakah aksi uni bisa dilakukan
48	cekTangkap(N,_,N):- !. cekTangkap(I,Count,_):- urutanGiliran(R1), get_element(R1, I, C), listpemain(C, _, X, _), jumlahElemenList(X, Sum), ceksatu(Sum), !, write(Count), write('. tangkap'), nl. cekTangkap(I,Count,N):- I1 is I+1, cekTangkap(I1,Count,N).	Memeriksa apakah aksi tangkap bisa dilakukan

49	lihatCommand:- urutanGiliran(R1), top_card(A, B), jumlahPemain(N), get_element(R1, 0, C), listpemain(C, _, X, Y), nl, write('Aksi utama yang tersedia:'), nl, cekMain(A,B,X,Y,Count), write(Count), write('. ambilKartu'), nl, C1 is Count+1, cekTantang(A, C1, O1), cekUni(X,O1,O2), cekTangkap(1,O2,N), nl, write('Aksi pendukung yang tersedia:'), nl, write('1. lihatCommand'), nl, write('2. lihatKartu'), nl, write('3. cekInfo'), nl.	Memperlihatkan aksi-aksi yang bisa dilakukan saat ini